



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 2: Swarm Intelligence

Dr Pedro Machado pedro.machado@ntu.ac.uk



Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Ant Colony Optimisation (ACO)
- Artificial Bee Colony (ABC)
- Bird Swarm Algorithm (BSA)
- Firefly Algorithm (FA)
- Artificial Fish Swarm Algorithm (AFSA)
- Whale Optimisation Algorithm (WOA)
- Dolphin Partner Optimisation (DPO)

- Cuckoo Search (CS)
- Grey Wolf Optimiser (GWO)
- Bat Algorithm (BA)
- Grasshopper Optimisation Algorithm (GOA)
- Shark Smell Optimisation (SSO)
- Particle Swarm Optimisation (PSO)

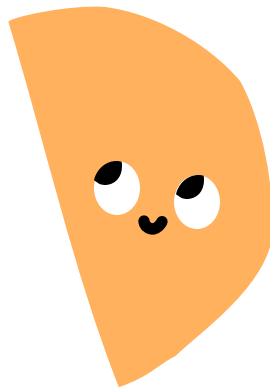
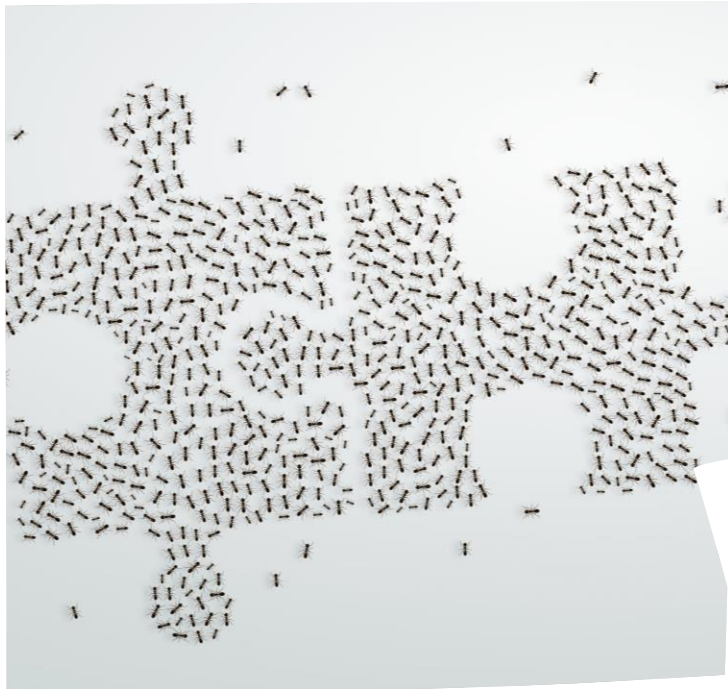
This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

Understanding Swarm Intelligence



Concept of Swarm Intelligence

SI is the collective behaviour of decentralised, self-organised agents inspired by natural swarms like ants and bees.

Emergent Problem Solving

Simple local interactions lead to complex global behaviour enabling efficient problem solving without central control.

Robustness and Adaptability

SI systems are fault tolerant, scalable, and adaptive, maintaining function despite agent failures in dynamic environments.

Applications of SI

SI principles inspire algorithms solving optimisation problems in engineering, logistics, AI, and telecommunications.

Ant Colony Optimisation (ACO)

Natural Collective Behaviours

Simple organisms like ants, bees, birds, and fish display complex group behaviours through local interactions.

Core Swarm Intelligence Principles

Decentralisation, local interactions, and emergent global patterns enable efficient and adaptive collective actions.

Advantages of Swarm Systems

Swarm systems offer robustness, scalability, and flexibility, adapting to failures and environmental changes.

Algorithmic Inspirations

Algorithms like ACO mimic ant and bee behaviours to solve optimisation problems efficiently.



ACO - Biological Basis and Stigmergy



Stigmergy in Ant Behaviour

Ants communicate indirectly by depositing pheromones, guiding others to optimal paths through environmental modification.

Ant Colony Optimisation Model

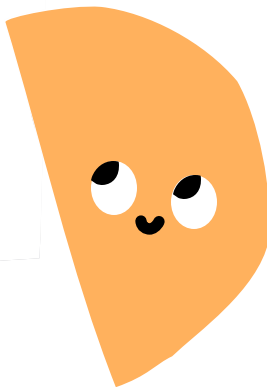
ACO uses artificial ants that probabilistically construct solutions based on pheromone intensity and heuristic information.

Pheromone Evaporation and Reinforcement

Evaporation reduces pheromone on less optimal routes while reinforcement amplifies promising paths, balancing exploration and exploitation.

Applications and Adaptability

ACO's decentralised and adaptive nature suits dynamic problems like network routing and solving NP-hard optimisation tasks.



ACO Structure and Parameter Tuning

Core Algorithm Components

ACO consists of solution construction, pheromone updates, and optional daemon actions for optimisation.

Parameter Influence

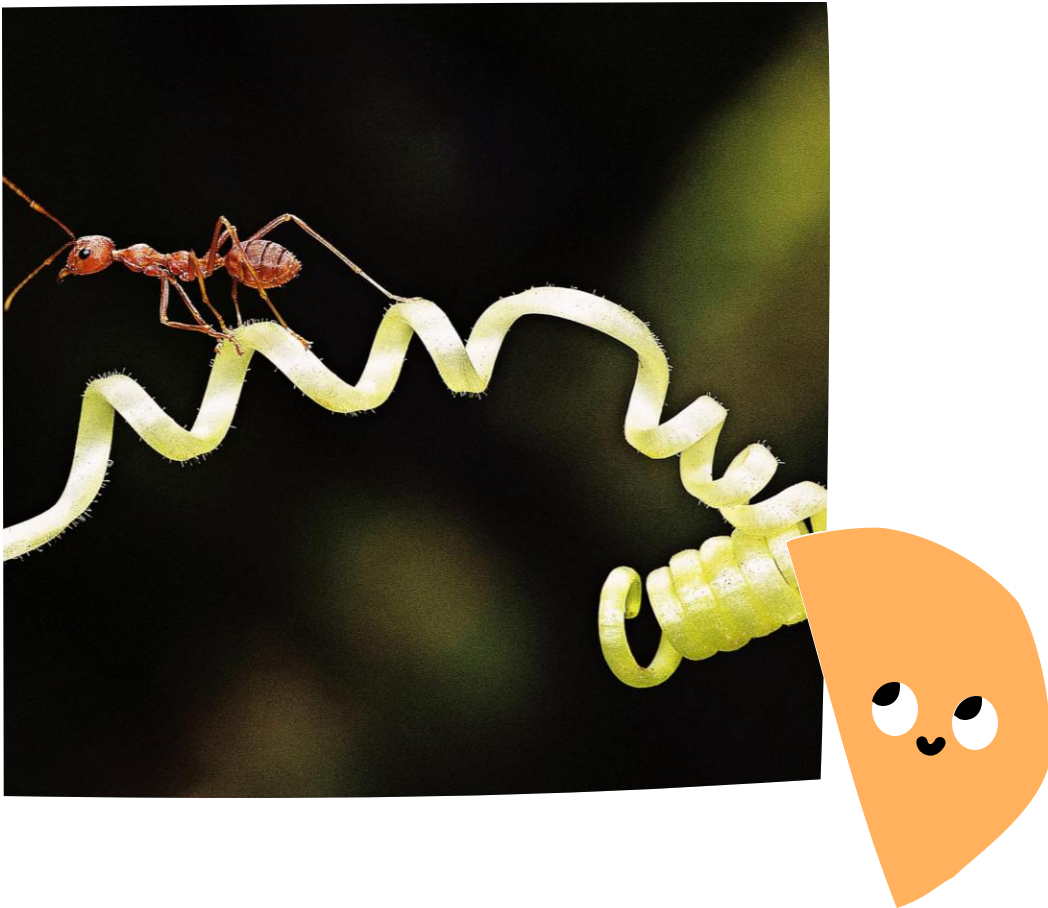
Parameters α and β control pheromone and heuristic importance, balancing exploration and exploitation.

Pheromone Update Mechanism

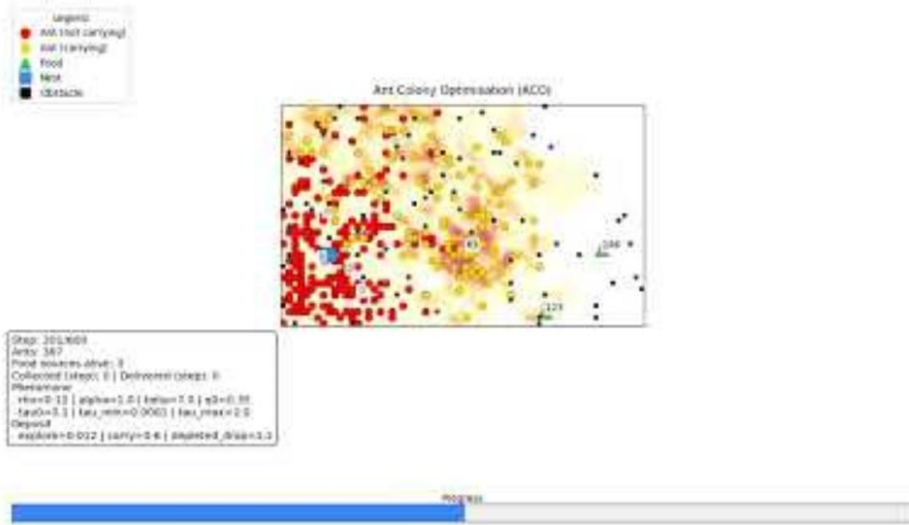
Pheromone evaporation and reinforcement guide ants toward shorter paths while maintaining diversity.

Advanced Parameter Tuning

Adaptive control and hybrid methods improve convergence and solution quality in complex problems.



ACO equation and Simulation



An ant k at node r moves to node s with probability:

$$P_{r,s}^k = \frac{\tau(r,s)^\alpha \cdot \eta(r,s)^\beta}{\sum_{u \in J_k(r)} \tau(r,u)^\alpha \cdot \eta(r,u)^\beta}$$

- $\tau(r, s)$: Pheromone level on edge (r, s)
- $\eta(r, s)$: Heuristic desirability (e.g., inverse of distance)
- α : Importance of pheromone trail
- β : Importance of heuristic information
- $J_k(r)$: Set of nodes not yet visited by ant k

Bee Colony Optimisation (BCO)

Biological Inspiration

ABC optimisation mimics honeybees' foraging and communication via waggle dance for resource allocation.

Algorithm Agents

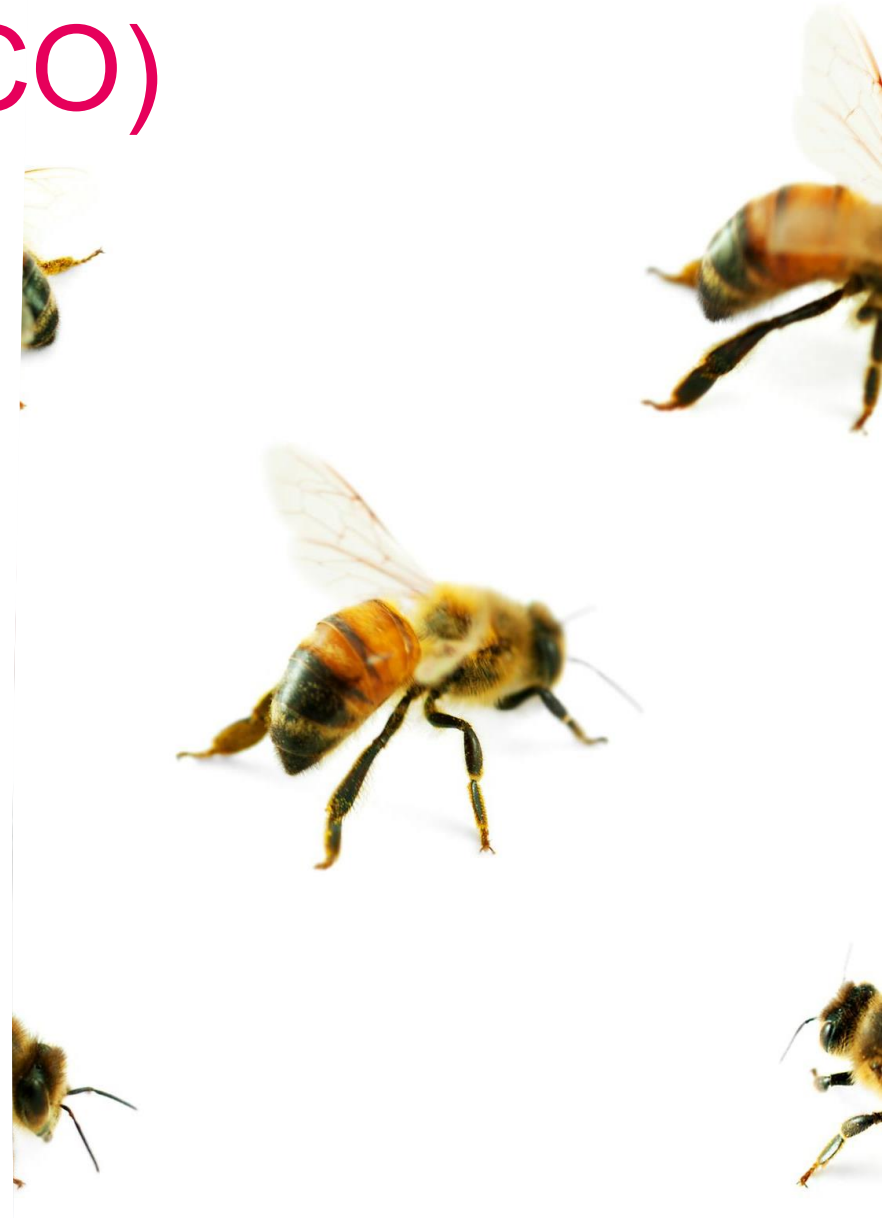
Three agent types (employed, onlooker, and scout bees) model exploration and exploitation in ABC.

Optimisation Process

ABC iteratively updates solutions by neighborhood search and uses fitness to guide exploration.

Algorithm Benefits

ABC is simple, adaptable, and excels in continuous optimisation with applications in clustering and design.



BCO - Core behaviours

1. Employed Bee Behaviour (Exploitation)

- a. Exploits the current food source by searching locally for a better neighboring solution.
- b. Generates a new candidate by modifying the current solution.
- c. Uses greedy selection - if the new solution is better, it replaces the old one else the old solution is retained.

2. Onlooker Bee Behaviour (Selection & Intensification)

- a. Observe the “waggle dance” information from employed bees.
- b. Select food sources probabilistically, favoring higher-quality solutions.
- c. After selection, perform local search around the chosen food source, similar to employed bees.

3. Scout Bee Behaviour (Exploration)

- a. When a food source does not improve for a predefined number of trials (limit), it is abandoned.
- b. The associated employed bee becomes a scout.
- c. The scout generates a completely new random solution in the search space.

4. Food Source Evaluation (Fitness Assessment)

- a. Each food source is evaluated using a fitness function that measures solution quality.
- b. Fitness influences the onlooker bee selection probability and decision to keep or abandon a food source

5. Information Sharing (Indirect Communication)

- a. Bees do not communicate directly via messages.
- b. Information is shared indirectly through fitness values and selection probabilities

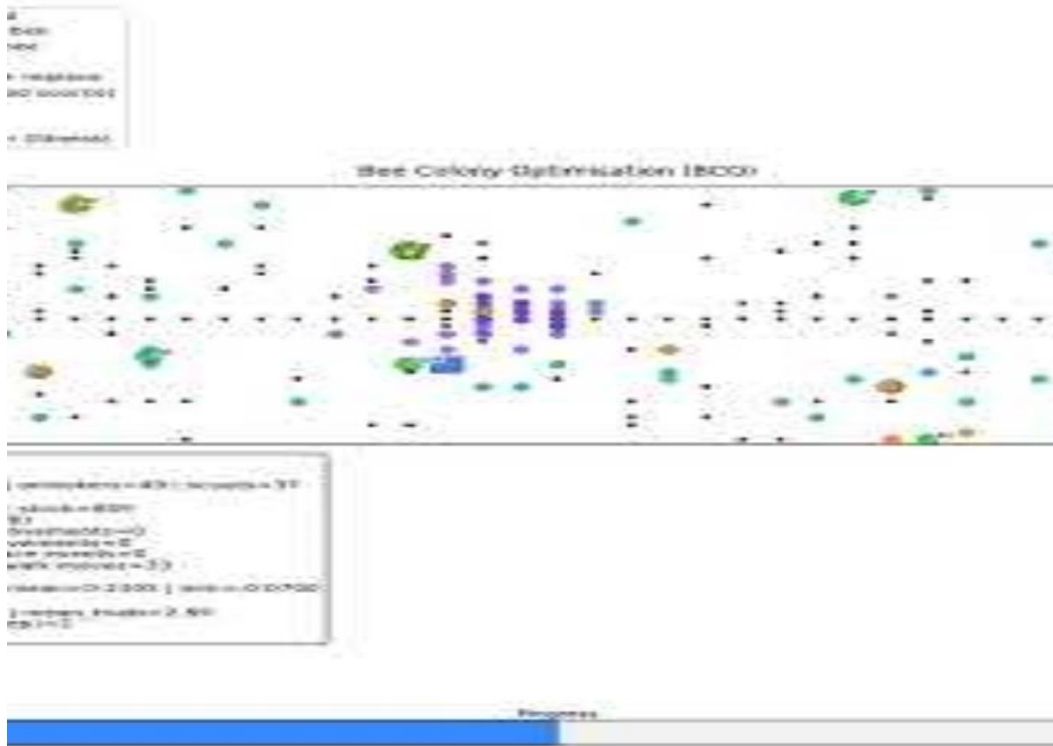
BCO - Probabilistic Path Selection

- Each ant chooses the next node based on:

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \in \text{allowed}} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}}$$

- τ_{ij} : pheromone level on edge $i \rightarrow j$
- η_{ij} : heuristic desirability (e.g., inverse of distance)
- α, β : control influence of pheromone vs heuristic
- This balances **exploration** (randomness) and **exploitation** (following strong trails).

BCO - Pheromone Update and simulation



- After completing a tour, ants deposit pheromone on edges they traveled:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{\text{ants}} \Delta\tau_{ij}$$

- ρ : evaporation rate (prevents unlimited accumulation)
- $\Delta\tau_{ij}$: pheromone added, often inversely proportional to tour length
- Evaporation ensures **old paths fade**, encouraging adaptation.

What is Bird Swarm Algorithm (BSA)?

Bio-Inspired Optimisation

BSA mimics social behaviours of bird flocks like foraging and vigilance to solve complex optimisation tasks efficiently.

Exploration and Exploitation

The algorithm balances global exploration and local exploitation to find optimal solutions in diverse problem landscapes.

Application Areas

BSA is applied in engineering design, scheduling, and energy management, showing versatility in optimisation challenges.

Algorithm Simplicity

BSA requires fewer control parameters and achieves rapid convergence with robustness across various tasks.



BSA – Core behaviours

Foraging Behaviour

Birds search for food using personal and shared information, modeled as updates to best solutions in the algorithm.

Vigilance Behaviour

Birds monitor peers and adjust positions to avoid threats, enhancing diversity and preventing premature convergence.

Flight (Migration) Behaviour

Periodic relocation of the flock helps escape local optima, maintaining global search capability and exploration.



BSA - Position Update Rules

Foraging Phase Updates

Birds update positions using personal and global bests with learning coefficients and randomness for exploitation.

Vigilance Phase Dynamics

Birds move towards swarm mean and better individuals with randomness to maintain diversity and avoid stagnation.

Migration and Exploration

Birds periodically relocate or reinitialise randomly to explore promising regions and enhance search diversity.

Boundary Handling

Algorithm ensures solutions stay within feasible limits by applying boundary constraints during updates.



BSA - Foraging Behaviour (search for food)

Each bird updates its position based on its own historical best and the global best:

$$X_i^{t+1} = X_i^t + \phi_1 \cdot (P_i^t - X_i^t) + \phi_2 \cdot (G^t - X_i^t)$$

- X_i^t : position of bird i at iteration t
- P_i^t : personal best position of bird i
- G^t : global best position in the swarm
- $\phi_1, \phi_2 \sim U(0, 1)$: random coefficients controlling influence of personal and global best

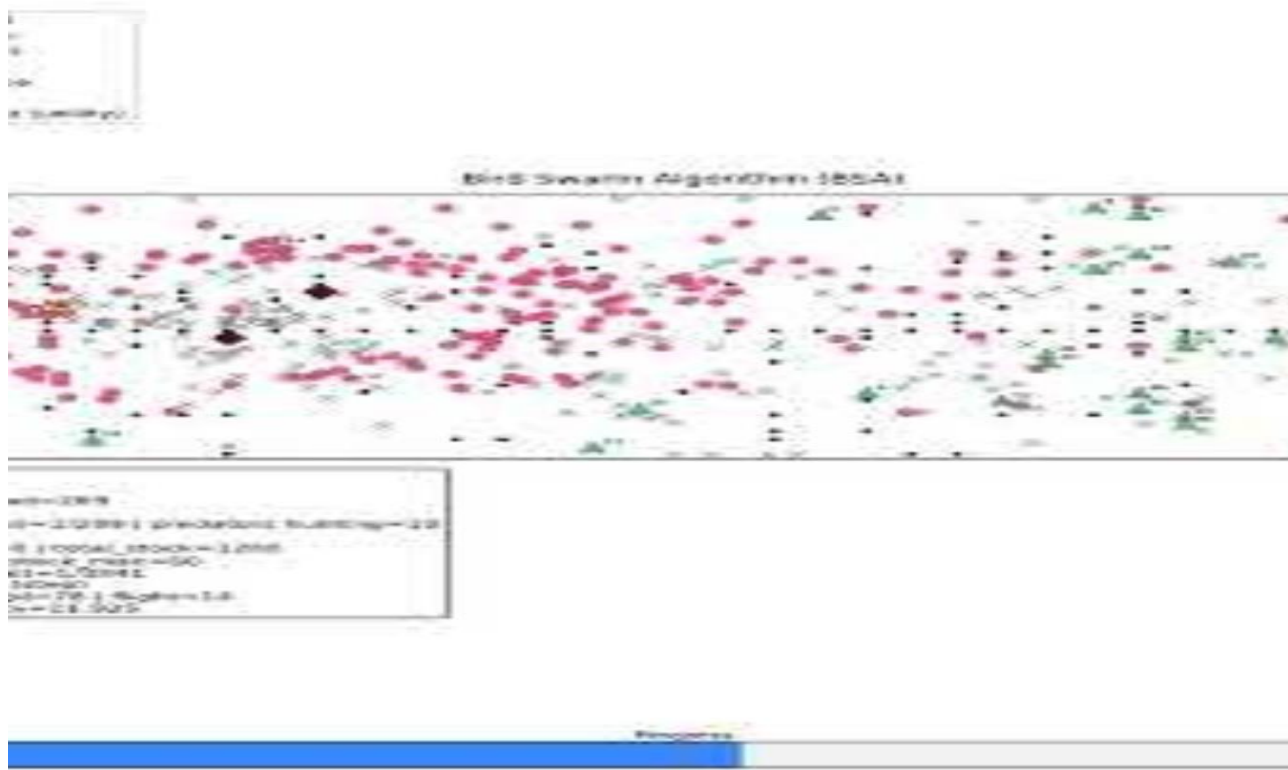
BSA - Vigilance Behaviour (group awareness)

Birds adjust toward the **mean position** and better individuals:

$$X_i^{t+1} = X_i^t + \alpha \cdot (M^t - X_i^t) + \beta \cdot (X_j^t - X_i^t)$$

- $M^t = \frac{1}{N} \sum_{k=1}^N X_k^t$: mean position of the swarm|
- X_j^t : position of a randomly selected bird with better fitness
- $\alpha, \beta \sim U(0, 1)$: random weights

BSA - Flight/Migration Behaviour and Simulation



After a certain number of iterations, birds relocate to escape local optima:

$$X_i^{t+1} = X_i^t + \gamma \cdot (X_k^t - X_i^t)$$

or random reinitialization:

$$X_i^{t+1} = L_b + r \cdot (U_b - L_b)$$

- X_k^t : randomly chosen bird
- L_b, U_b : lower and upper bounds of the search space
- $r \sim U(0, 1)$: random number
- γ : migration coefficient

Fitness and Selection

- Birds keep track of their **personal best** and update the **global best** after each iteration.
- Switching between behaviours is controlled by probabilities (e.g., foraging vs vigilance).

Firefly Algorithm (FA)

Luciferin-Based Attraction

Agents use luciferin values to attract neighbors, updating values based on decay and enhancement constants for fitness.

Dynamic Neighborhood Formation

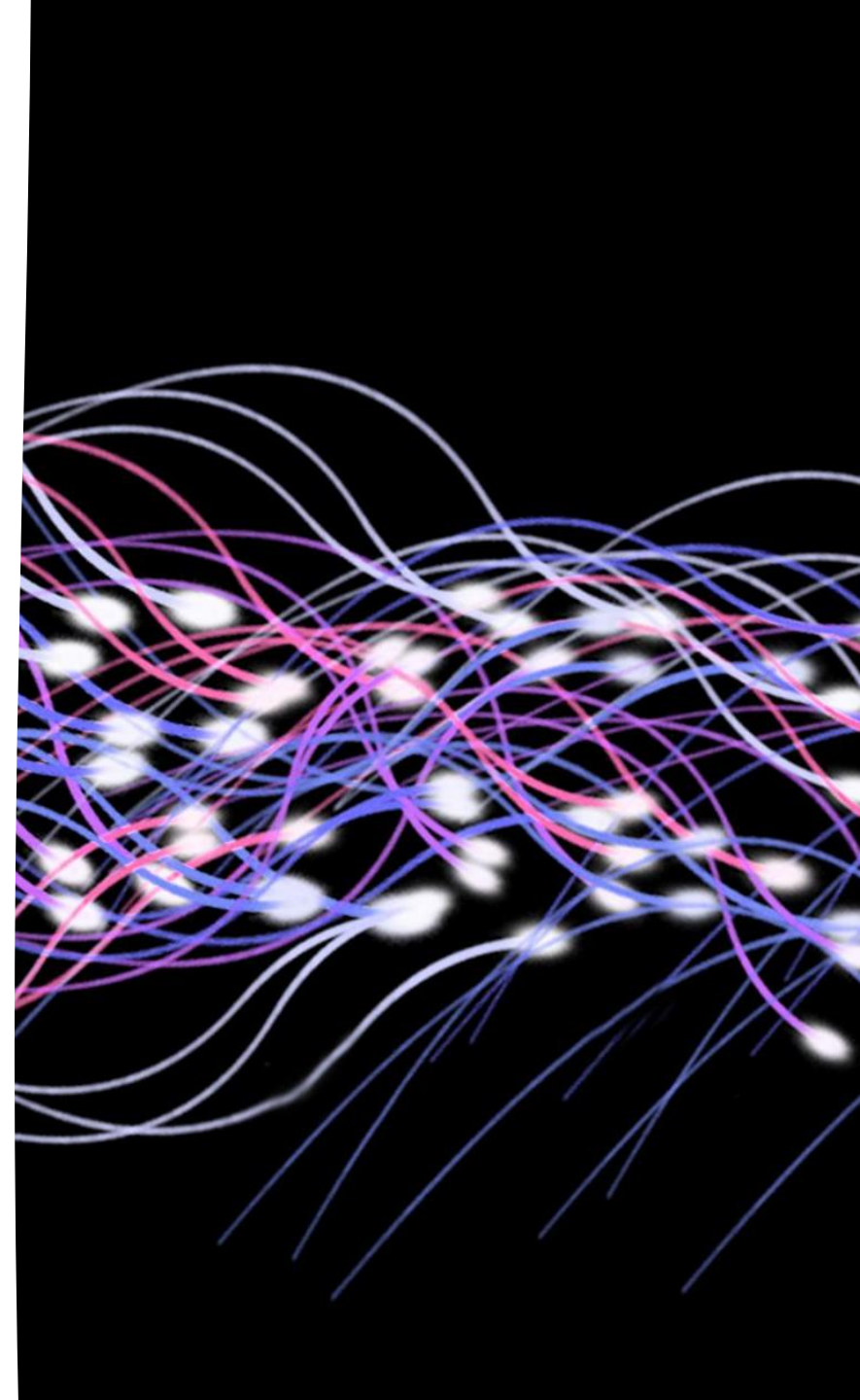
Agents dynamically select neighbors within range, moving towards those with higher luciferin to maintain diversity.

Multimodal Optimisation Benefits

GSO forms multiple subgroups around different optima, offering robustness and adaptability in complex problems.

Applications and Limitations

GSO is applied in robotics, sensor deployment, and clustering but has slower convergence in high dimensions.



FA – Behaviours

- Bioluminescent Attraction
- Distance-Dependent Attractiveness
- Random Movement
- Brightness Linked to Fitness

FA - Attractiveness Function

The attractiveness of a firefly decreases with distance due to light absorption:

$$\beta(r) = \beta_0 e^{-\gamma r^2}$$

- β_0 : attractiveness at $r = 0$ (maximum)
- γ : light absorption coefficient
- r : distance between two fireflies

FA - Distance Between Fireflies and Position Update Rule

For two fireflies i and j at positions x_i and x_j :

$$r_{ij} = \|x_i - x_j\|$$

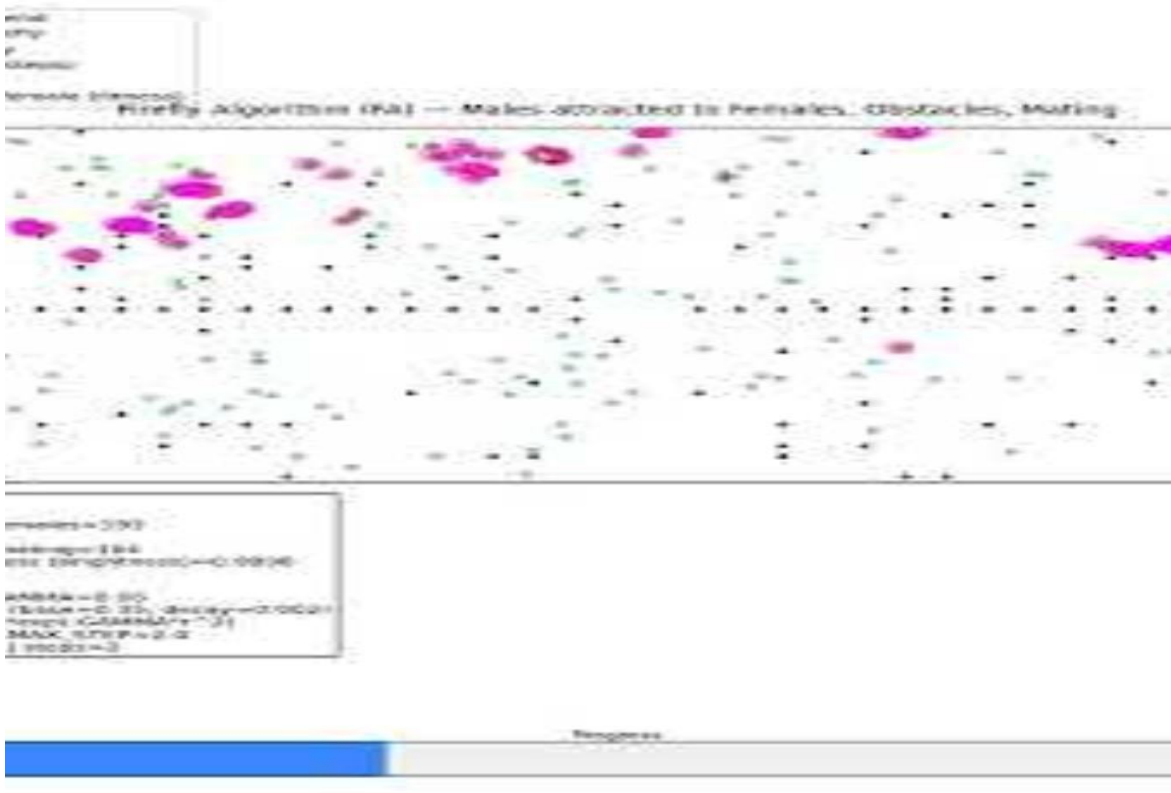
Usually Euclidean distance in the search space.

If firefly i is less bright (worse fitness) than firefly j , it moves toward j :

$$x_i^{t+1} = x_i^t + \beta_0 e^{-\gamma r_{ij}^2} (x_j^t - x_i^t) + \alpha \epsilon$$

- x_i^t : position of firefly i at iteration t
- $\beta_0 e^{-\gamma r_{ij}^2} (x_j^t - x_i^t)$: deterministic attraction toward brighter firefly j
- $\alpha \epsilon$: randomization term (ϵ is a random vector, often uniform in $[-0.5, 0.5]$)
- α : randomization parameter controlling exploration

FA - Brightness (Light Intensity) and Simulation



Brightness is linked to the objective function $f(x)$. For minimization:

$$I_i \propto \frac{1}{f(x_i) + \delta}$$

or simply $I_i = f(x_i)$ for maximization problems.

Artificial Fish Swarm Algorithm (AFSA)

Prey Mode Exploration

Prey mode enables random local search to discover better solutions by mimicking fish preying behaviour.

Swarm Mode Cohesion

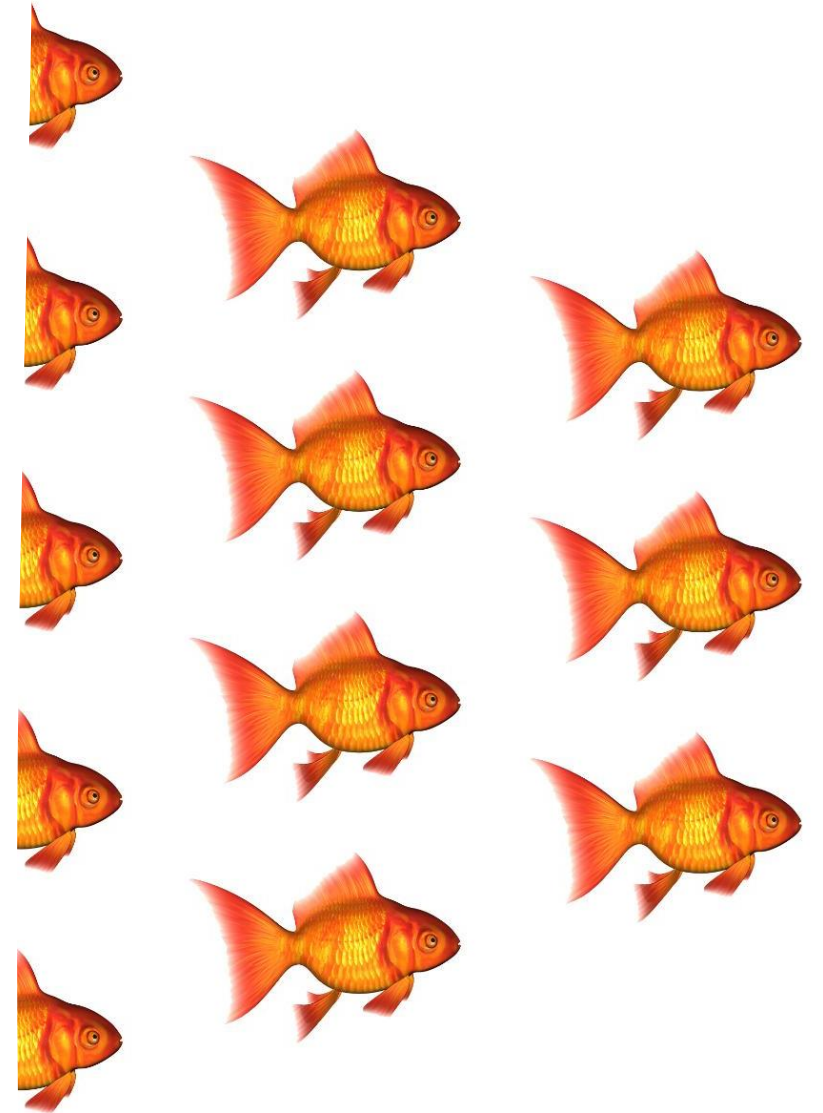
Swarm mode drives fish toward neighbors' center to maintain group coherence and share information.

Follow Mode Optimisation

Follow mode guides fish to move toward better-performing neighbors to improve solution quality.

Algorithm Strengths and Applications

AFSA offers strong global search and robustness, applied in image segmentation and engineering optimisation.



AFSA - Behaviours

- Preying Behaviour: Represents fish searching for food randomly within its visual range.
- Swarming Behaviour: Fish tend to gather around the centre of the swarm while avoiding overcrowding.
- Following Behaviour: Fish follow a neighbor with better fitness within its visual range.
- Random Behaviour: If no better position is found during preying, swarming, or following, the fish moves randomly.

AFSA - Preying Behaviour

Fish searches randomly within its visual range and moves toward a better position.

Candidate generation

$$\mathbf{X}_j = \mathbf{X}_i + \text{Visual} \cdot (2 \cdot \text{rand}() - 1)$$

Movement update

If

$$f(\mathbf{X}_j) > f(\mathbf{X}_i)$$

then

$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \text{rand}() \cdot \text{Step} \cdot \frac{\mathbf{X}_j - \mathbf{X}_i}{\|\mathbf{X}_j - \mathbf{X}_i\|}$$

Else repeat search up to `Try_number` times.

Notation

Symbol	Meaning
$\mathbf{X}_i(t)$	Position of fish i at iteration t
$f(\mathbf{X})$	Fitness (objective function)
Visual	Visual distance (neighbourhood radius)
Step	Maximum movement length
$\delta \in (0, 1]$	Crowd factor
n_f	Number of fish
$\mathcal{N}_i$	Set of neighbours within Visual
$\mathbf{X}_c$	Centre position of neighbours
$\mathbf{X}_j$	A neighbour with better fitness
$\text{rand}()$	Uniform random number in $[0, 1]$

AFSA - Following Behaviour

Fish follows the **best neighbour** in its visual range.

Best neighbour

$$\mathbf{X}_j = \arg \max_{\mathbf{X}_k \in \mathcal{N}_i} f(\mathbf{X}_k)$$

Condition

If

$$f(\mathbf{X}_j) > f(\mathbf{X}_i) \quad \text{and} \quad \frac{|\mathcal{N}_i|}{n_f} < \delta$$

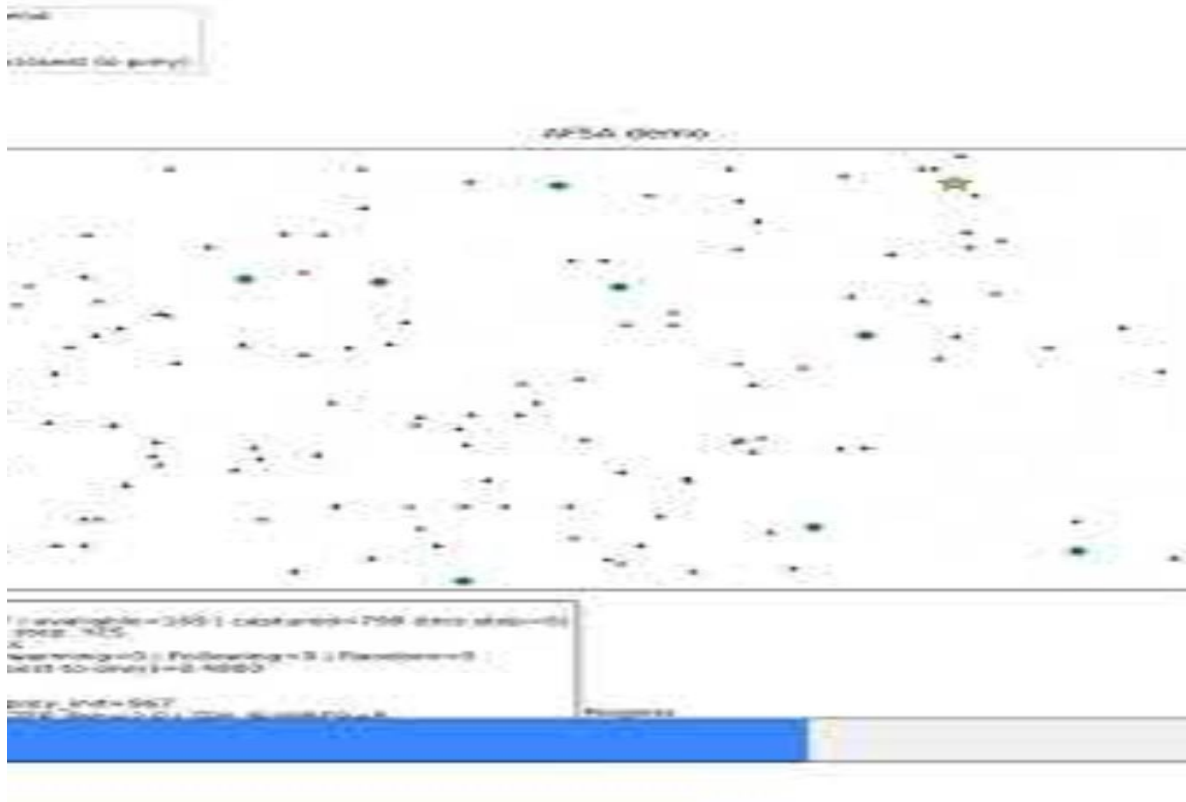
Movement

$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \text{rand}() \cdot \text{Step} \cdot \frac{\mathbf{X}_j - \mathbf{X}_i}{\|\mathbf{X}_j - \mathbf{X}_i\|}$$

Notation

Symbol	Meaning
$\mathbf{X}_i(t)$	Position of fish i at iteration t
$f(\mathbf{X})$	Fitness (objective function)
Visual	Visual distance (neighbourhood radius)
Step	Maximum movement length
$\delta \in (0, 1]$	Crowd factor
n_f	Number of fish
$\mathcal{N}_i$	Set of neighbours within Visual
$\mathbf{X}_c$	Centre position of neighbours
$\mathbf{X}_j$	A neighbour with better fitness
rand()	Uniform random number in $[0, 1]$

AFSA - Random Behaviour and simulation



If **preying, swarming, and following** all fail, fish moves randomly:

$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \text{Step} \cdot (2 \cdot \text{rand}() - 1)$$

Notation

Symbol	Meaning
$\mathbf{X}_i(t)$	Position of fish i at iteration t
$f(\mathbf{X})$	Fitness (objective function)
Visual	Visual distance (neighbourhood radius)
Step	Maximum movement length
$\delta \in (0, 1]$	Crowd factor
n_f	Number of fish
$\mathcal{N}_i$	Set of neighbours within Visual
$\mathbf{X}_c$	Centre position of neighbours
$\mathbf{X}_j$	A neighbour with better fitness
$\text{rand}()$	Uniform random number in $[0, 1]$

Whale Optimisation Algorithm (WOA)

- **Encircling Prey Mechanism** - WOA models encircling prey using distance and coefficient vectors to update candidate solutions effectively.
- **Spiral Updating Mechanism** - Spiral updating simulates whales' spiral motion using exponential and cosine functions for solution refinement.
- **Algorithm Balance and Applications** - WOA balances exploration and exploitation, suited for optimisation in feature selection and engineering design.



WOA – Core behaviours

WOA alternates between encircling (direct approach) and spiral movement (bubble-net attack) to balance global and local search.

- Encircling Prey - Whales assume the prey is at the best-known position and encircle it.
- Spiral Update Mechanism - bubble-net attack using a logarithmic spiral around the prey
- Behaviour Switching - With probability p , whales choose encircling or spiral update.

WOA - Spiral Update Mechanism

- Models the **bubble-net attack** using a logarithmic spiral around the prey.
- Position update:

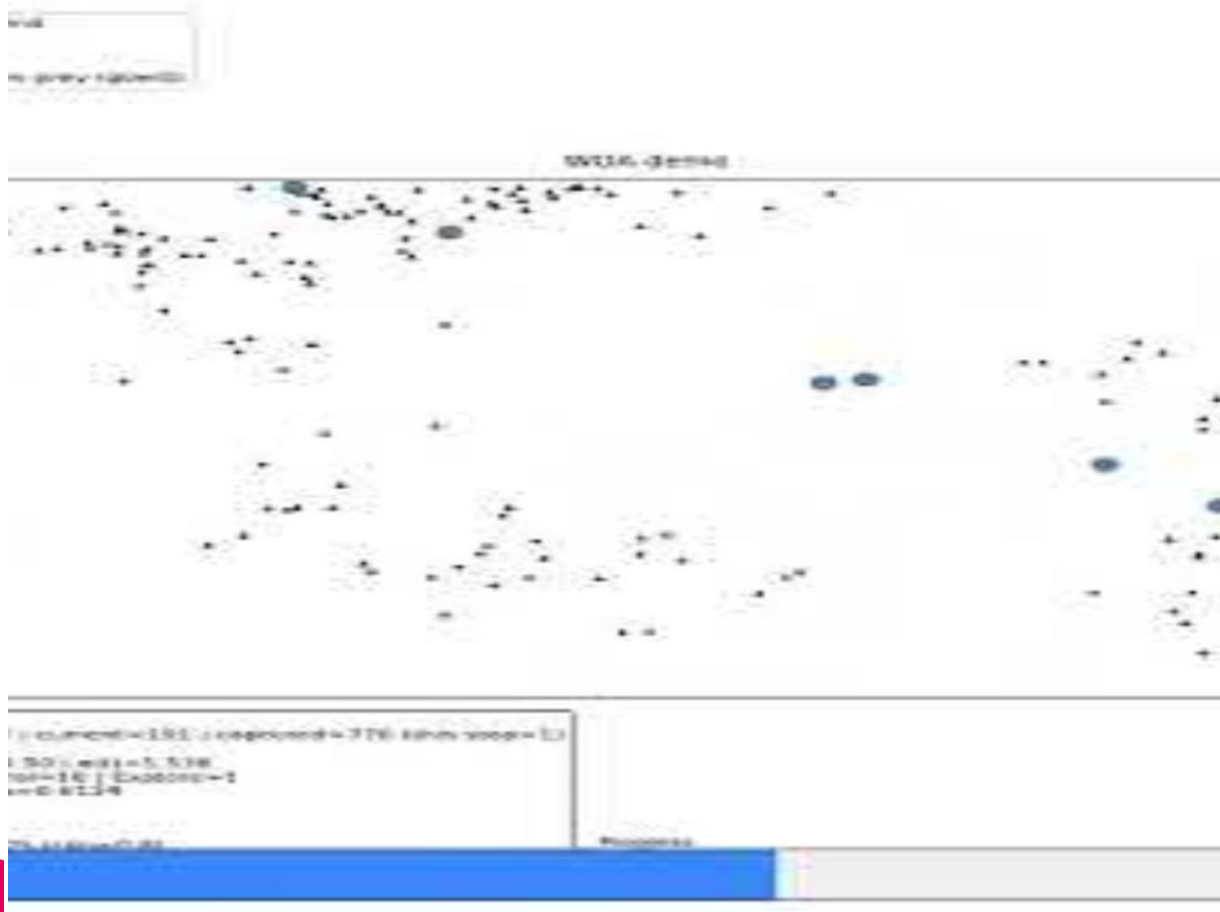
$$\mathbf{X}^{t+1} = \mathbf{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \mathbf{X}^*$$

Where:

- $\mathbf{D}' = |\mathbf{X}^* - \mathbf{X}|$
- b : constant defining spiral shape
- $l \in [-1, 1]$: random number

Purpose: Exploration and exploitation — whales spiral toward prey while searching around it.

WOA - Encircling Prey and simulation



- Whales assume the prey is at the best-known position and **encircle** it.
- Position update:

$$D = |C \cdot X^* - X|$$

$$X^{t+1} = X^* - A \cdot D$$

Where:

- X^* : current best solution (prey position)
- X : current whale position
- $A = 2a \cdot r - a$, $C = 2r$ (coefficients)
- a : linearly decreases from 2 to 0 over iterations
- $r \sim U(0, 1)$: random number

Purpose: Exploitation — move closer to the best solution.

- With probability p , whales choose **encircling** or **spiral** update.
- Random search occurs when $|A| > 1$ (exploration mode).

Dolphin Partner Optimisation (DPO)

Cooperative Hunting Inspiration

The algorithm mimics dolphins' cooperative hunting by emphasising partner selection and information sharing.

Position Adjustment Mechanism

Each agent updates its position based on partner location and global best solution without velocity vectors.

Diversity Maintenance

Periodic regrouping and partner reassignment prevent stagnation and maintain population diversity.

Strengths and Applications

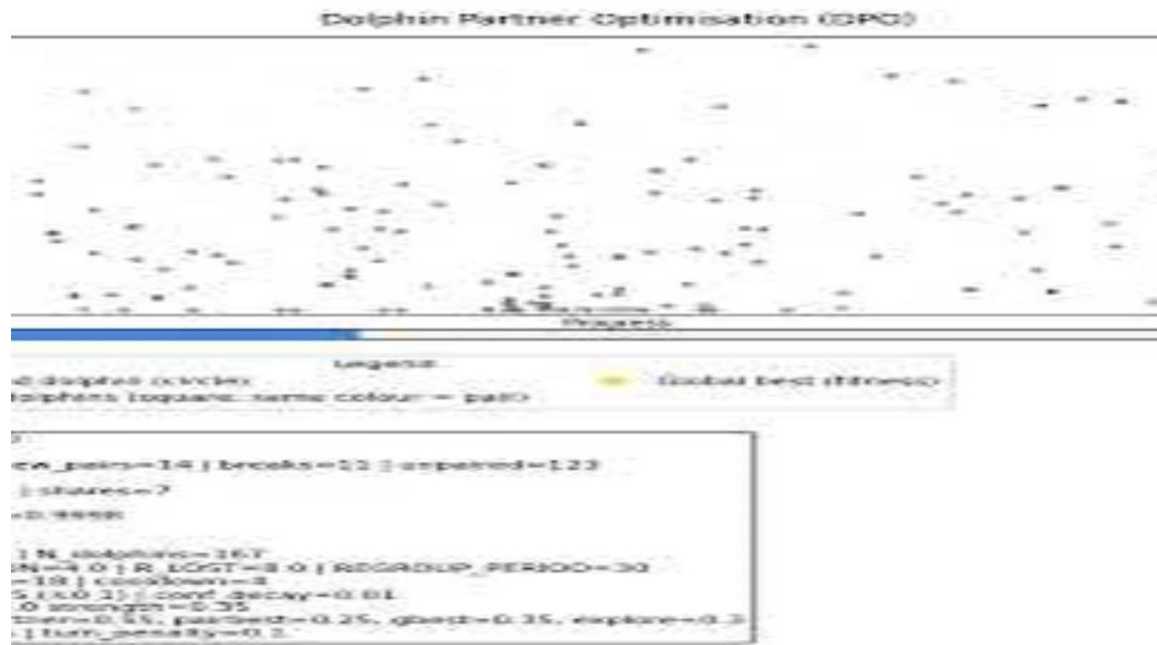
Strengths include improved convergence and adaptability; applications include scheduling and routing.



DPO – Core behaviours

- Partner Selection - form pairs or small groups to hunt efficiently.
- Cooperative Movement - adjust their positions toward their partner and the best-known solution.
- Regrouping - after several iterations, dolphins reassign partners or form new groups.
- Information Sharing - share knowledge of good hunting spots (solutions) with partners.

DPO - Cooperative Movement and simulation



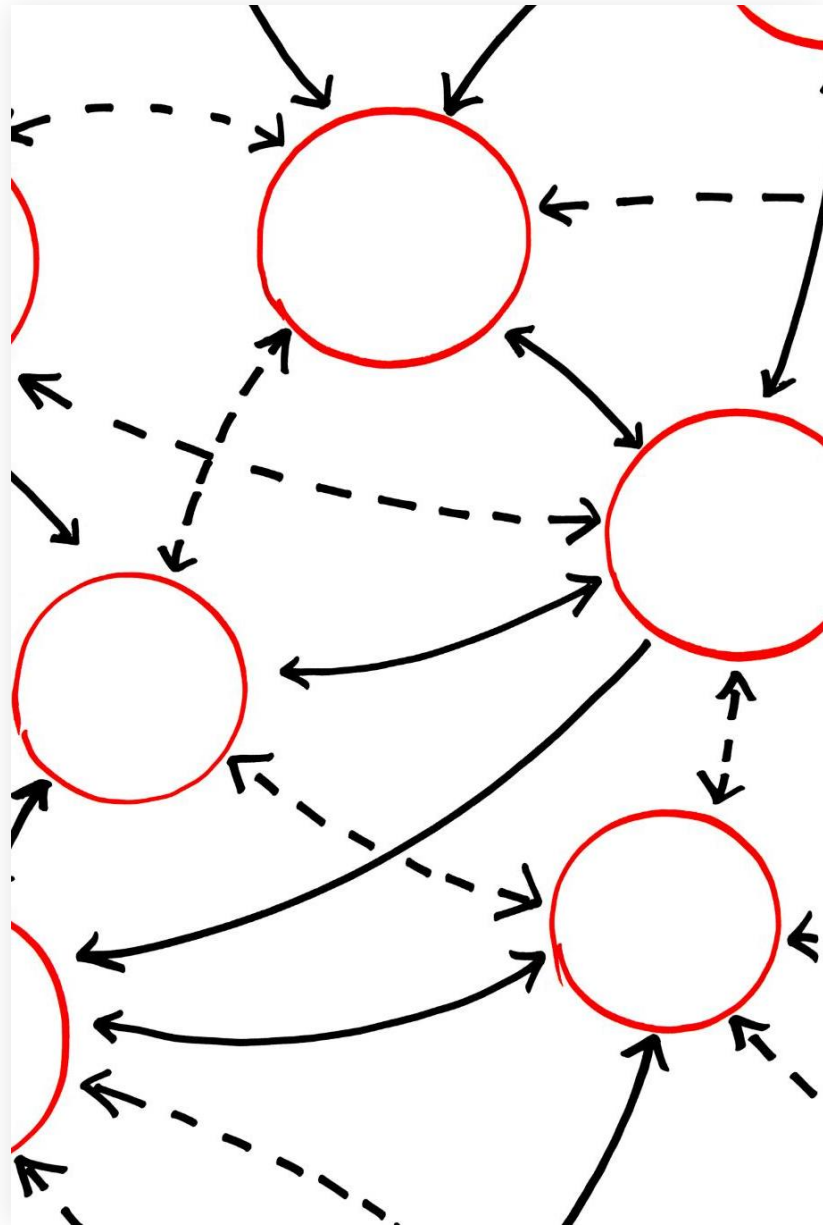
- Dolphins adjust their positions toward their partner and the best-known solution.
- Movement rule often combines:

$$X_i^{t+1} = X_i^t + r_1(Partner - X_i^t) + r_2(Best - X_i^t)$$

where $r_1, r_2 \sim U(0, 1)$.

- Balances **exploration** (partner influence) and **exploitation** (best solution).

DPO emphasizes **cooperation** and **dynamic partner switching** to explore the search space effectively while exploiting promising regions.



Cuckoo Search (CS)

Global Exploration - enable long-tailed random steps for effective global search in optimisation algorithms like Cuckoo Search.

Nest Replacement Strategy - A fraction of the worst nests is replaced randomly to maintain diversity and avoid premature convergence.

Strengths and Applications - Cuckoo Search excels in global search and is applied in engineering, neural networks, and image processing tasks.

CS – Core behaviours

- Lévy Flight Exploration (Cuckoo egg laying) - Each cuckoo generates a new candidate solution by performing a Lévy flight from its current position.
- Host Nest Selection (Random nest to compare) - A cuckoo's new solution is evaluated against a randomly chosen nest.
- Elitism / Keep Best Solutions - The best nests (solutions) are always retained and carried forward.
- Discovery & Abandonment (Host detects alien eggs)
- Iterative Selection to Convergence

CS - equations and simulation

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

Grey Wolf Optimiser (GWO)

Leadership Hierarchy - GWO models alpha, beta, delta, and omega wolves to simulate leadership in optimisation.

Encircling Mechanism - Encircling uses distances and candidate positions to update solutions iteratively.

Exploration and Exploitation - Coefficients A and C control the balance between global and local search phases.

Applications and Challenges - GWO is effective for optimisation but may stagnate without diversity strategies; used in feature selection and power systems.



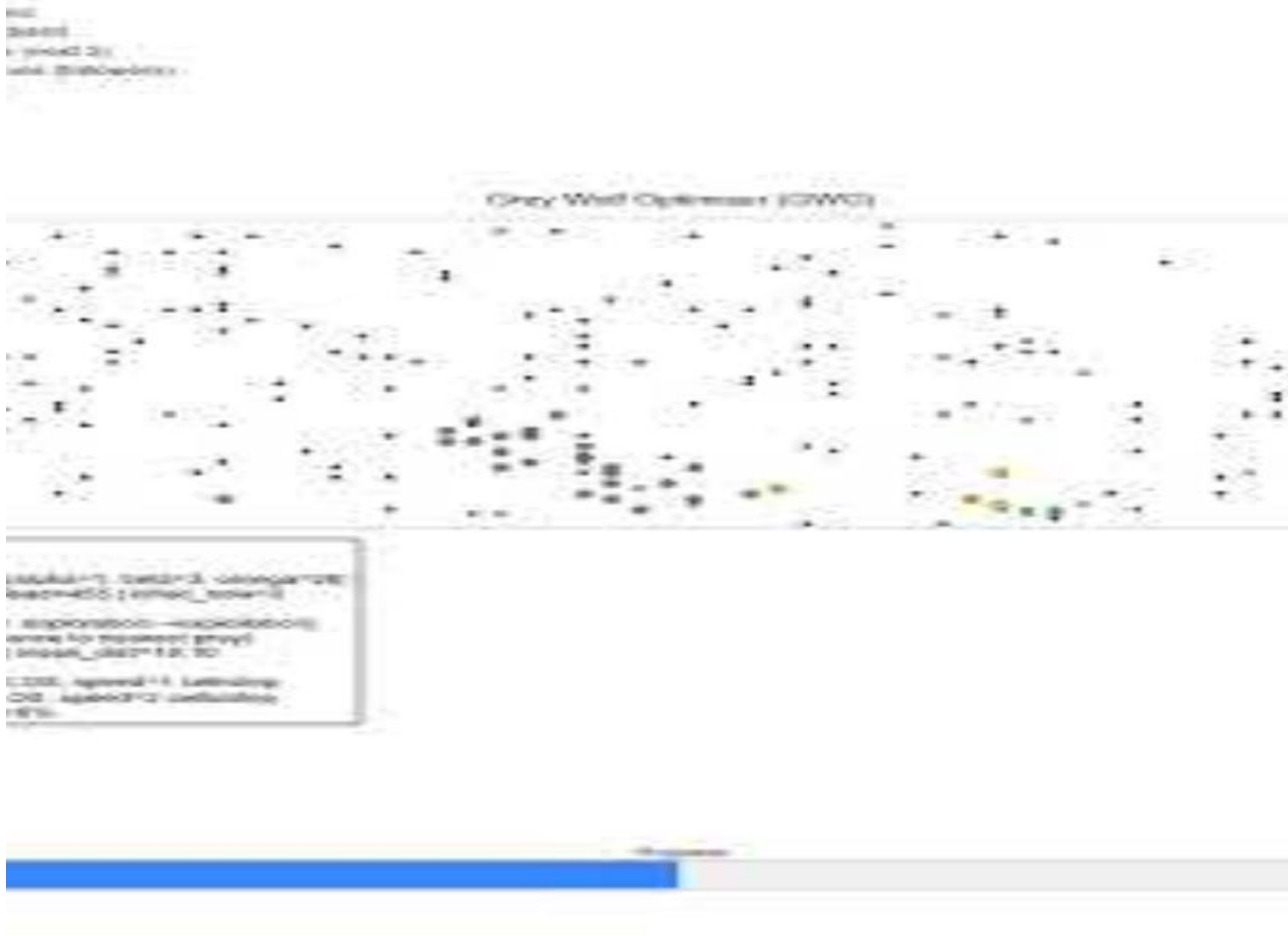
GWO - behaviours

- Social Hierarchy - wolves are ranked as:
 - Alpha (α): Leader, best solution.
 - Beta (β): Second-best solution.
 - Delta (δ): Third-best solution.
 - Omega (ω): Remaining wolves.

The top three wolves guide the search, while omegas follow.

- Encircling Prey
- Hunting (Position Update)
- Exploration vs Exploitation - Controlled by parameter α :
 - Early iterations: α large $\rightarrow$ exploration (wolves spread out).
 - Later iterations: α small $\rightarrow$ exploitation (wolves converge to prey).

WOA - Encircling prey, hunting and simulation



- Wolves estimate the prey's position (best solution) and encircle it:

$$D = |C \cdot X^* - X|$$

$$X^{t+1} = X^* - A \cdot D$$

Where:

- X^* : prey (best solution)
- $A = 2a \cdot r_1 - a$, $C = 2r_2$
- a : decreases linearly from 2 to 0 (controls exploration/exploitation)
- $r_1, r_2 \sim U(0, 1)$
- Wolves update positions based on α , β , and δ :

$$X_1 = X_\alpha - A_1 \cdot |C_1 \cdot X_\alpha - X|$$

$$X_2 = X_\beta - A_2 \cdot |C_2 \cdot X_\beta - X|$$

$$X_3 = X_\delta - A_3 \cdot |C_3 \cdot X_\delta - X|$$

$$X^{t+1} = \frac{X_1 + X_2 + X_3}{3}$$

Bat Algorithm (BA)

- **Bio-Inspired Algorithm Concept** - Bat Algorithm mimics echolocation behaviour to explore and exploit solution space effectively.
- **Position and Velocity Updates** - Positions and velocities update dynamically using frequency tuning similar to particle swarm optimisation.
- **Exploration to Exploitation Shift** - Loudness decreases and pulse rate increases to transition from global search to local refinement.
- **Applications and Parameters** - Widely used in optimisation and engineering with key parameters like population size and frequency bounds.



Core Behaviours of Bat Algorithm

Frequency Tuning - Adjusts sensing resolution and step size for broad exploration and later refinement.

Velocity and Position Updates - Incorporates momentum and attraction towards the global best to exploit promising regions.

Pulse-Loudness Adaptation - Dynamically decreases loudness and increases pulse rate, focusing search locally over time.

Local Random Walks - Small perturbations near quality solutions help escape local optima effectively.



BA - Frequency and velocity update

Each bat adjusts its frequency to control step size:

$$f_i = f_{\min} + (f_{\max} - f_{\min}) \cdot \beta$$

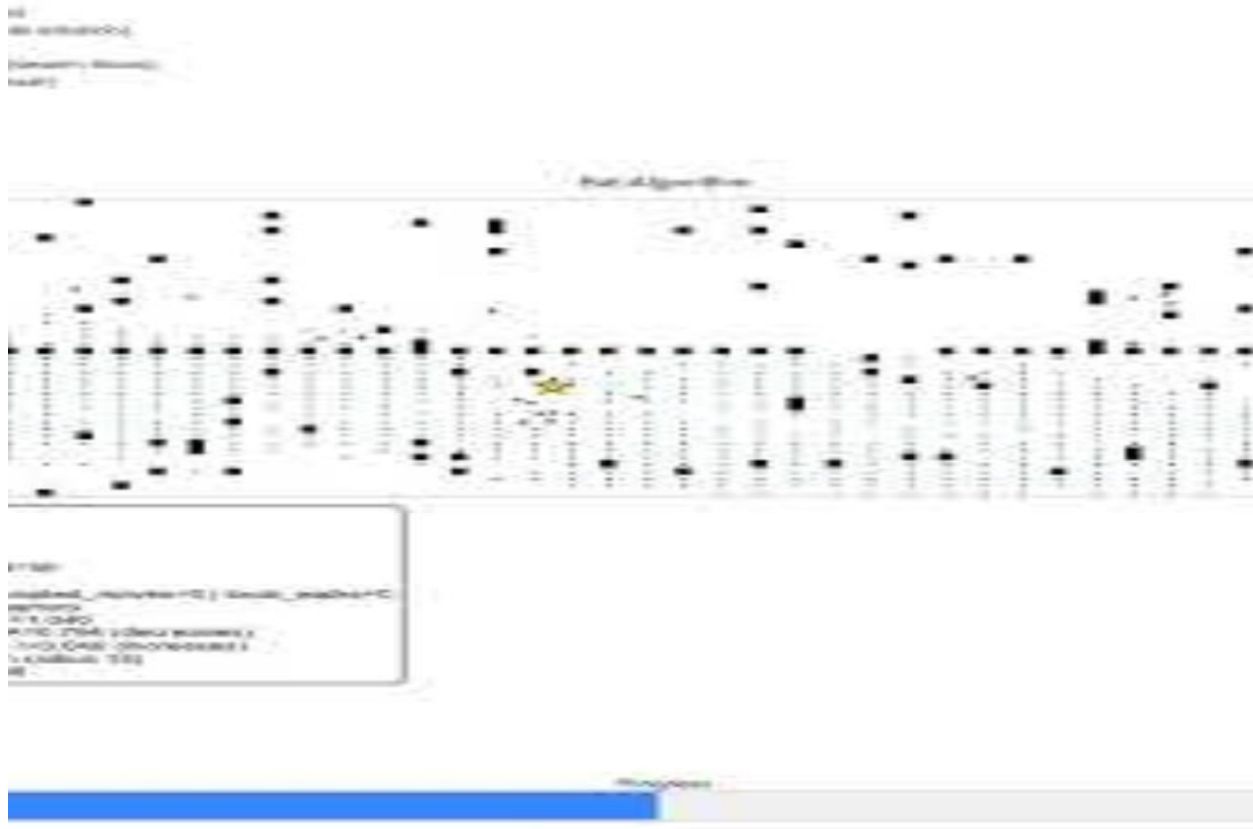
- f_i : frequency of bat i
- $f_{\min}, f_{\max}$: minimum and maximum frequency bounds
- $\beta \sim U(0, 1)$: random number

Velocity depends on the difference between the bat's position and the global best:

$$v_i^{t+1} = v_i^t + (x_i^t - gbest) \cdot f_i$$

- v_i^t : velocity of bat i at iteration t
- x_i^t : current position of bat i
- $gbest$: global best position found so far

BA - Position update, loudness and pulse rate adaptation and simulation



Position is updated using the new velocity:

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

To balance exploration and exploitation:

$$A_i^{t+1} = \alpha \cdot A_i^t, \quad r_i^{t+1} = r_i^0 \cdot [1 - e^{-\gamma t}]$$

- A_i : loudness (decreases over time)
- r_i : pulse emission rate (increases over time)
- α, γ : constants controlling adaptation

Grasshopper Optimisation Algorithm (GOA)

Swarming Behaviour Inspiration

GOA is inspired by grasshopper swarming and migration behaviours, modelling social interactions for optimisation.

Dynamic Position Update

Position updates combine social interaction, gravity, and wind, balancing exploration and exploitation dynamically.

Exploration and Exploitation Balance

Early iterations explore globally with large interaction ranges; later focus narrows for local exploitation.

Practical Applications

GOA is applied in engineering, scheduling, and machine learning due to its adaptive search capabilities.



GOA - Core Behaviours

Repulsion and Attraction Forces - Forces maintain spacing among individuals while allowing clustering near promising regions to prevent premature convergence.

Environmental Drift Effect - Gravity and wind terms bias movement to ensure directional progress and avoid stagnation in the search process.

Contraction Schedule Dynamics - Gradual reduction of parameter c shifts the algorithm from global roaming to focused local refinement over time.

Target Bias and Constraint Handling - Individuals are pulled toward the best solution while keeping positions within feasible bounds via projection or reflection.



GOA - Repulsion & attraction forces and environmental drift effect

The social interaction among grasshoppers is modeled by:

$$\mathbf{S}_i = \sum_{j \neq i} s(d_{ij}) \cdot \hat{d}_{ij}$$

Where:

- $\mathbf{S}_i$: social force acting on grasshopper i
- $d_{ij} = \|X_j - X_i\|$: distance between grasshopper i and j
- $\hat{d}_{ij} = \frac{X_j - X_i}{d_{ij}}$: unit direction vector
- $s(r)$: social interaction function:

$$s(r) = f \cdot e^{-r/l} - e^{-r}$$

with:

- f : attraction intensity
- l : attractive length scale

This ensures **repulsion at short distances** and **attraction at moderate distances**, maintaining spacing while clustering near promising regions.

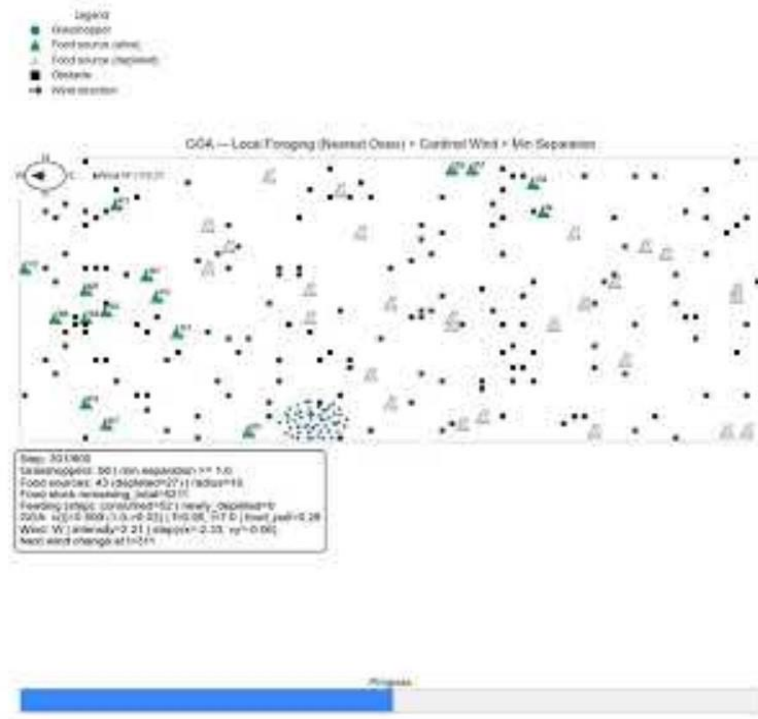
Position update includes gravity and wind terms:

$$X_i^{t+1} = c \cdot \mathbf{S}_i + \mathbf{g} + \mathbf{u}$$

Where:

- $\mathbf{g}$: gravity vector (bias toward ground or target)
- $\mathbf{u}$: wind advection vector (directional drift) These terms **bias movement** to ensure progress and avoid stagnation.

GOA - Contraction schedule dynamics and target bias & constraint Handling and simulation



The contraction coefficient c decreases linearly over iterations:

$$c = c_{\max} - \frac{(c_{\max} - c_{\min}) \cdot t}{T}$$

Where:

- $c_{\max}, c_{\min}$: initial and final values of c
- t : current iteration
- T : maximum iterations

This gradually shifts the algorithm from **global roaming** to **local refinement**.

Grasshoppers are pulled toward the best solution while respecting bounds:

$$X_i^{t+1} = \text{clip}(X_i^{t+1}, L_b, U_b)$$

Where:

- L_b, U_b : lower and upper bounds
- $\text{clip}()$ or reflection ensures feasibility

Additionally, the best solution influences the swarm through the social term and environmental drift.



Shark Smell Optimisation

Algorithm Inspiration - SSO is inspired by sharks' chemotaxis-driven hunting and group coordination strategies in nature.

Movement Modeling - Movement combines gradient-following with momentum and stochastic perturbations to guide search effectively.

Exploration and Diversity - Random swims and reinitialisation maintain diversity and prevent stagnation during optimisation.

Key Parameters and Strengths - Parameters include momentum coefficient, gradient weight, noise scale, and restart policy balancing search dynamics.

Core Behaviours of Shark Smell Optimisation

Chemotaxis Gradient Tracking - SSO mimics shark chemotaxis to track fitness gradients, steering towards optimal solution regions effectively.

Inertia and Smooth Pursuit - Inertia preserves previous velocity in SSO, enabling smooth pursuit and faster convergence during optimisation.

Random Swim for Diversity - Random swim introduces stochastic perturbations to escape local minima and maintain diversity in search.

Schooling and Cooperation - Schooling behaviour enables sharing of best directions, promoting cooperative movement and information exchange.

Greedy Update and Adaptation - Greedy update rules accept improvements immediately, while unsuccessful moves trigger step damping or diversification.



SSO - Chemotaxis gradient tracking and Inertia and Smooth Pursuit

SSO mimics sharks following the strongest odor (fitness gradient):

$$\mathbf{v}_i^{t+1} = \lambda \mathbf{v}_i^t + \eta \nabla f(\mathbf{x}_i^t)$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + s \cdot \frac{\mathbf{v}_i^{t+1}}{\|\mathbf{v}_i^{t+1}\|}$$

Where:

- $\mathbf{x}_i^t$: position of shark i
- $\mathbf{v}_i^t$: velocity of shark i
- $\nabla f(\mathbf{x}_i^t)$: estimated fitness gradient
- λ : inertia factor
- η : gradient influence coefficient
- s : step size

Preserve part of previous velocity for smooth movement:

$$\mathbf{v}_i^{t+1} = \lambda \mathbf{v}_i^t + \dots$$

This term ensures continuity and accelerates convergence.

SSO - Random Swim for Diversity and Schooling and Cooperation

Introduce stochastic perturbations to escape local minima:

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^{t+1} + \sigma \boldsymbol{\xi}$$

Where:

- σ : noise scale
- $\boldsymbol{\xi} \sim U(-1, 1)^D$: random vector This maintains diversity and prevents stagnation.

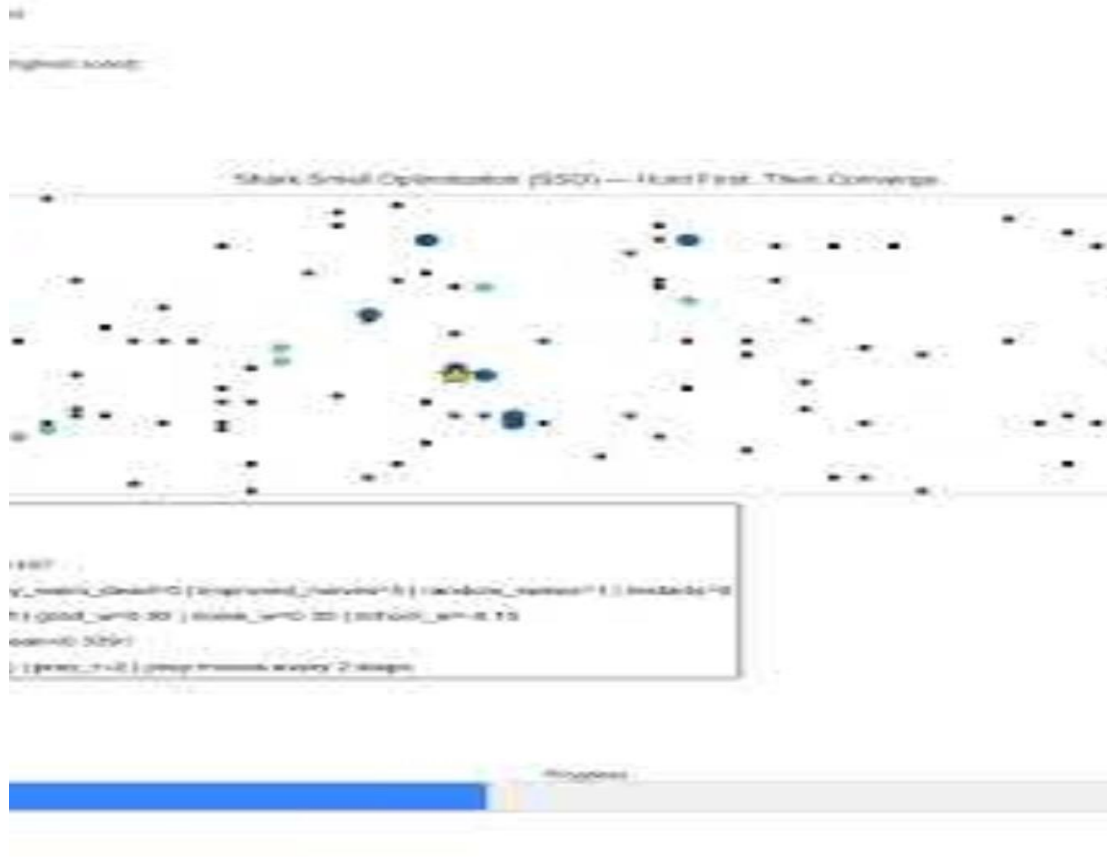
Incorporate shared best directions:

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^{t+1} + \delta(\mathbf{x}^* - \mathbf{x}_i^t)$$

Where:

- $\mathbf{x}^*$: global best position
- δ : cooperation coefficient This promotes collective movement toward promising regions.

SSO - Greedy Update & Adaptation and simulation



Update position only if improvement occurs:

If $f(\mathbf{x}_i^{t+1}) < f(\mathbf{x}_i^t)$ then accept; else damp step or diversify.

Damping example:

$$s \leftarrow \alpha s, \alpha < 1$$

This ensures adaptive step control and diversification when stuck.

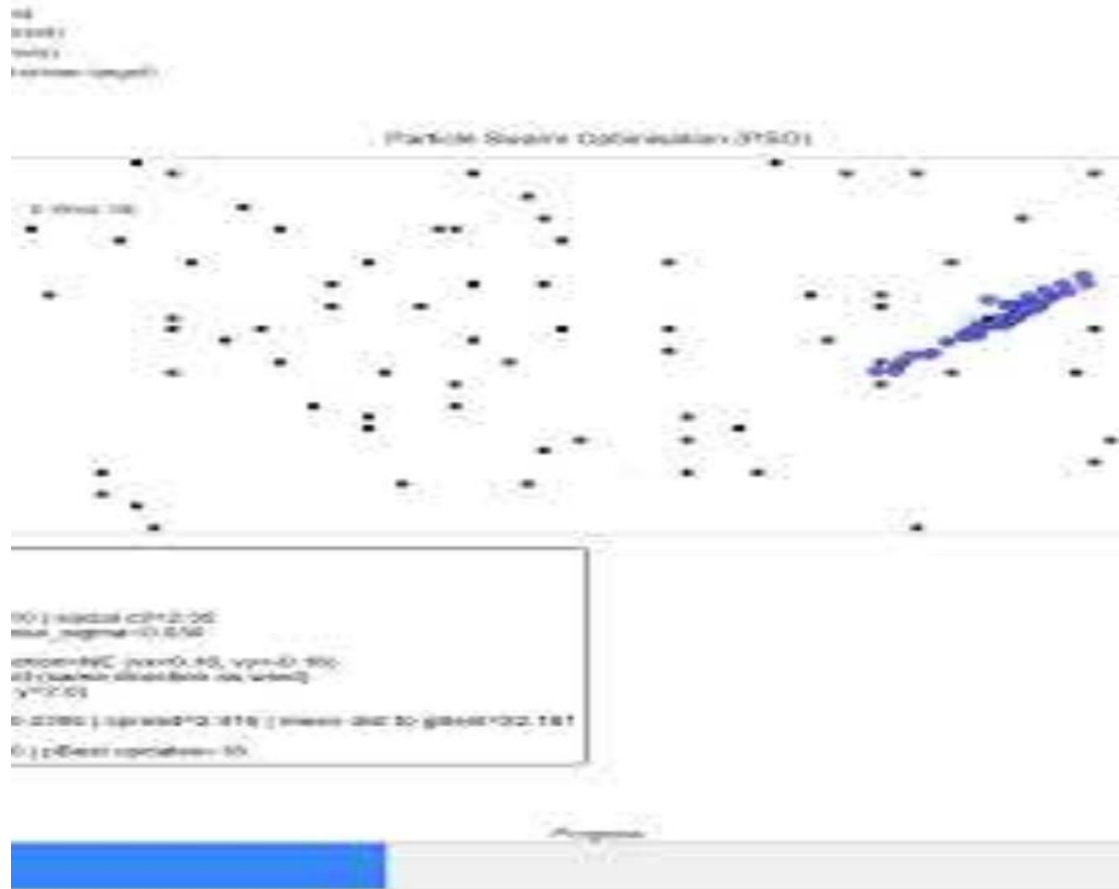
Particle Swarm Optimisation

- population-based, stochastic optimisation algorithm inspired by the collective motion of birds and fish.
- A population of candidates, called particles, moves through the search space
- Each particle represents a potential solution
- Particles adjust their movement based on:
 - Their own experience
 - The experience of other particles in the swarm
- The algorithm iteratively improves solutions by updating positions and velocities
- Key characteristics:
 - Simple to implement
 - Few parameters
 - Effective for continuous, non-linear optimisation problems

PSO - Core Behaviours

- PSO is driven by three fundamental behaviours:
- 1. Inertia (Momentum)
 - Particles tend to continue moving in their current direction
 - Encourages exploration of the search space
- 2. Cognitive Behaviour (Self-Learning)
 - Each particle is attracted toward its personal best solution (pBest)
 - Represents individual experience and memory
- 3. Social Behaviour (Social Learning)
 - Particles are attracted toward the best solution found by the swarm (gBest)
 - Enables information sharing and cooperation

PSO Update Equations



- Velocity Update

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pBest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gBest - x_i(t))$$

- Position Update

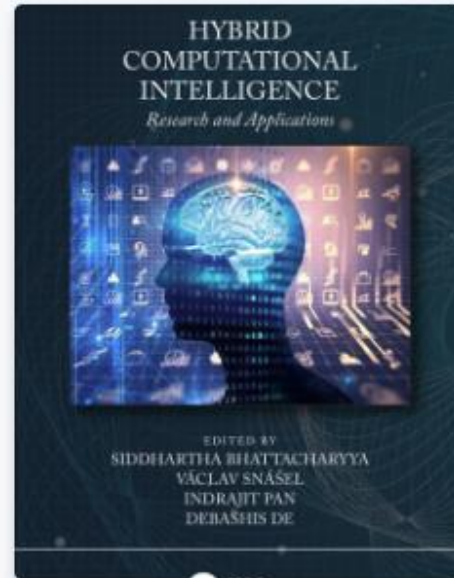
$$x_i(t+1) = x_i(t) + v_i(t+1)$$

- Where:

- v_i = velocity of particle i
- x_i = position of particle i
- w = inertia weight
- c_1, c_2 = cognitive and social coefficients
- r_1, r_2 = random numbers in $[0,1]$
- $pBest_i$ = best solution found by particle i

Algorithm	Target Problems	Real-Life Applications
Ant Colony Optimisation (ACO)	Combinatorial optimisation (routing, scheduling)	Network routing, vehicle routing, logistics, TSP
Artificial Bee Colony (ABC)	Continuous optimisation, clustering	Image segmentation, load balancing, structural design
Bird Swarm Algorithm (BSA)	Mixed optimisation tasks, dynamic landscapes	Energy management, scheduling, engineering optimisation
Firefly Algorithm (FA)	Multimodal optimisation, nonlinear problems	Image processing, clustering, machine learning parameter tuning
Artificial Fish Swarm Algorithm (AFSA)	Global optimisation, multimodal search	Image segmentation, water resource management, engineering design
Whale Optimisation Algorithm (WOA)	Continuous optimisation, feature selection	Power system optimisation, structural design, medical diagnosis
Dolphin Partner Optimisation (DPO)	Cooperative search, scheduling	Job-shop scheduling, routing, resource allocation
Cuckoo Search (CS)	Global optimisation, metaheuristic search	Engineering design, neural networks, image processing
Grey Wolf Optimiser (GWO)	Continuous optimisation, feature selection	Renewable energy optimisation, machine learning, power systems
Bat Algorithm (BA)	Continuous optimisation, dynamic problems	Wireless sensor networks, image processing, engineering design
Grasshopper Optimisation Algorithm (GOA)	Large-scale optimisation, nonlinear problems	Machine learning, scheduling, structural optimisation
Shark Smell Optimisation (SSO)	Gradient-based optimisation, dynamic search	Chemical process optimisation, robotics path planning
Particle Swarm Optimisation (PSO)	Continuous optimisation, high-dimensional search spaces	Neural network training, feature selection, engineering design

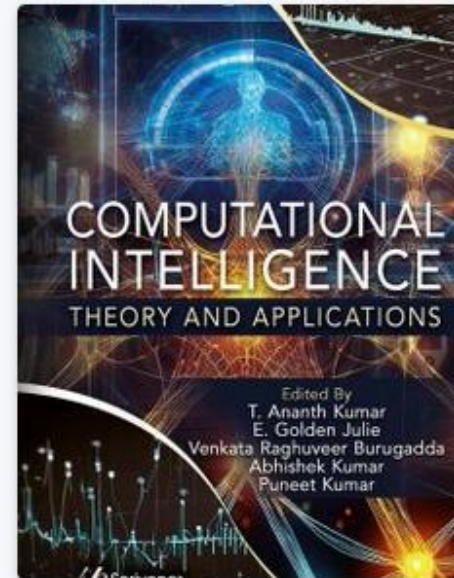
References



eBook

...

**Hybrid Computational
Intelligence**



eBook

...

Computational Intelligence



Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- Ant Colony Optimisation (ACO)
- Artificial Bee Colony (ABC)
- Bird Swarm Algorithm (BSA)
- Firefly Algorithm (FA)
- Artificial Fish Swarm Algorithm (AFSA)
- Whale Optimisation Algorithm (WOA)

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

- Dolphin Partner Optimisation (DPO)
- Cuckoo Search (CS)
- Grey Wolf Optimiser (GWO)
- Bat Algorithm (BA)
- Grasshopper Optimisation Algorithm (GOA)
- Shark Smell Optimisation (SSO)
- Particle Swarm Optimisation (PSO)



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 2: Swarm Intelligence

Dr Pedro Machado pedro.machado@ntu.ac.uk